

UNIT 5 – Python Packages

Python packages: Simple programs using the built-in functions of packages matplotlib, numpy, pandas etc.

GUI Programming: Tkinter introduction, Tkinter and PythonProgramming, Tk Widgets, Tkinter examples. Python programming with IDE.

1. What is a Python Package?

A **package** in Python is a **collection of modules (Python files)** that help us perform specific tasks easily without writing code from scratch.

◆ Some Common Packages

Package	Purpose
math	Mathematical functions
random	Random number generation
numpy	Numerical computing (arrays, matrices)
pandas	Data handling and analysis
matplotlib	Data visualization (graphs, charts)

Why Packages are Useful

- They **save time** (no need to code from zero).
- They are **reusable** and **well-tested**.
- Used heavily in **Data Science, Machine Learning, and Engineering Projects**.

How to Install a Package

Before using a package, we install it using **pip** (Python's package manager).

```
pip install numpy
pip install pandas
pip install matplotlib
```

Then we **import** them in Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2. NumPy (Numerical Python)

NumPy is used for **mathematical and numerical operations** — especially on **arrays** (like lists but faster).

Creating Arrays

```
import numpy as np

# create 1D array
a = np.array([1, 2, 3, 4, 5])
print(a)
```

Output:

```
[1 2 3 4 5]
```

Basic Functions

Function	Description	Example	Output
<code>np.array()</code>	Creates array	<code>np.array([1,2,3])</code>	<code>[1 2 3]</code>
<code>np.zeros(n)</code>	Creates array of zeros	<code>np.zeros(4)</code>	<code>[0. 0. 0. 0.]</code>
<code>np.ones(n)</code>	Creates array of ones	<code>np.ones(3)</code>	<code>[1. 1. 1.]</code>
<code>np.arange(start, stop, step)</code>	Range of numbers	<code>np.arange(1,10,2)</code>	<code>[1 3 5 7 9]</code>
<code>np.linspace(start, stop, n)</code>	n numbers evenly spaced	<code>np.linspace(0,10,5)</code>	<code>[0. 2.5 5. 7.5 10.]</code>

Mathematical Operations

```
import numpy as np

a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

print(a + b) # Addition
print(a - b) # Subtraction
print(a * b) # Multiplication
print(a / b) # Division
```

Output:

```
[11 22 33]
[ 9 18 27]
[10 40 90]
[10. 10. 10.]
```

Useful Functions

```
arr = np.array([5, 10, 15, 20])

print(np.min(arr))    # 5
print(np.max(arr))    # 20
print(np.mean(arr))   # 12.5
print(np.sum(arr))    # 50
```

✅ **Use in Data Science:** Used for **matrix**, **vector**, and **scientific calculations**.

3. Pandas (Data Analysis Library)

Pandas helps in **data manipulation and analysis**.

It provides two main data structures:

- **Series** → One-dimensional (like a column)
- **DataFrame** → Two-dimensional (like an Excel table)

Creating a Series

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

✅ **Index** on left, **data** on right.

Creating a DataFrame

```
import pandas as pd

data = {
    "Name": ["Gautam", "Ravi", "Ankit"],
    "Marks": [85, 90, 78],
    "City": ["Prayagraj", "Lucknow", "Kanpur"]
}

df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Marks	City
0	Gautam	85	Prayagraj
1	Ravi	90	Lucknow
2	Ankit	78	Kanpur

Common Pandas Functions

Function	Description	Example
df.head()	Shows first 5 rows	df.head()
df.tail()	Shows last 5 rows	df.tail()
df.info()	Shows structure of DataFrame	df.info()
df.describe()	Summary (mean, min, max)	df.describe()
df['column']	Access a single column	df['Marks']
df.to_csv('file.csv')	Save data to CSV file	—

Example: Reading CSV File

```
import pandas as pd

df = pd.read_csv("students.csv")
print(df.head())
```

✅ **Use in Data Science:** Pandas is used for **data cleaning, analysis, and storage**.

4. Matplotlib (Data Visualization Library)

Matplotlib is used to **plot graphs and charts** to visualize data easily.

We usually import it as:

```
import matplotlib.pyplot as plt
```

Basic Line Plot

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.title("Simple Line Graph")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Output: Line graph connecting points (1,10), (2,20), (3,25), (4,30)

Bar Chart Example

```
x = ['A', 'B', 'C', 'D']
y = [10, 20, 15, 25]

plt.bar(x, y, color='skyblue')
plt.title("Bar Chart Example")
plt.xlabel("Category")
plt.ylabel("Values")
plt.show()
```

Pie Chart Example

```
values = [40, 30, 20, 10]
labels = ['Python', 'C++', 'Java', 'C']

plt.pie(values, labels=labels, autopct='%1.1f%%')
plt.title("Programming Language Popularity")
plt.show()
```

Scatter Plot Example

```
x = [1, 2, 3, 4, 5]
y = [5, 10, 8, 12, 15]

plt.scatter(x, y, color='green')
plt.title("Scatter Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")

plt.show()
```

✅ **Use in Data Science:** Matplotlib helps visualize trends, correlations, and comparisons.

5. Combining NumPy, Pandas, and Matplotlib

All three libraries are often used together in data analysis.

Example:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Create random data
marks = np.random.randint(50, 100, 5)
names = ["A", "B", "C", "D", "E"]

# Make DataFrame
df = pd.DataFrame({"Name": names, "Marks": marks})
print(df)

# Plot the data
plt.bar(df["Name"], df["Marks"], color="orange")
plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.show()
```

Summary Table

Library	Purpose	Example Function
NumPy	Numerical operations	<code>np.array()</code> , <code>np.mean()</code>
Pandas	Data analysis & manipulation	<code>pd.DataFrame()</code> , <code>df.head()</code>
Matplotlib	Data visualization	<code>plt.plot()</code> , <code>plt.bar()</code>

UNIT 5 – Python Packages (Part 2)

1. Introduction to GUI Programming

☒ What is GUI?

GUI (Graphical User Interface) allows users to **interact with programs visually** — using buttons, text boxes, menus, etc., instead of typing commands.

Example in real life:

Calculator, Notepad, or Login Window are all GUI applications.

☒ Why GUI?

- Easy to use (no need to type commands)
- User-friendly interface
- More interactive and modern

2. Tkinter Introduction

Tkinter is the **standard GUI library** that comes with Python.

It helps us create desktop applications easily — like forms, calculators, or mini tools.
No need to install — Tkinter is built-in!

◇ **Importing Tkinter:**

```
import tkinter as tk
```

Or

```
from tkinter import *
```

3. Creating Your First Tkinter Window

Basic Structure:

```
import tkinter as tk

# Create main window
root = tk.Tk()

# Set title
root.title("My First GUI")

# Set window size
root.geometry("300x200")

# Run the window (main loop)
root.mainloop()
```

Explanation:

Line	Description
tk.Tk()	Creates the main window
title()	Sets window title
geometry()	Sets window size
mainloop()	Keeps window open until closed manually

Output:

A simple blank window titled “*My First GUI*”.

4. Tkinter Widgets (GUI Components)

Widgets are elements you see in a GUI — like **buttons**, **labels**, **text boxes**, etc.

(1) Label Widget

Used to display text or information.

```
from tkinter import *

root = Tk()
root.title("Label Example")
```

```
label1 = Label(root, text="Hello Gautam! Welcome to Tkinter.")
label1.pack()

root.mainloop()
```

Output:

A small window with text message “Hello Gautam! Welcome to Tkinter.”

(2) Button Widget

Used to perform an action when clicked.

```
from tkinter import *

def greet():
    print("Hello, GKM!")

root = Tk()
root.title("Button Example")

btn = Button(root, text="Click Me", command=greet)
btn.pack()

root.mainloop()
```

Output:

A button appears — when clicked, it prints “Hello, GKM!” in the terminal.

(3) Entry Widget

Used to take **input** from the user.

```
from tkinter import *

root = Tk()
root.title("Entry Example")

Label(root, text="Enter your name:").pack()
entry = Entry(root)
entry.pack()

root.mainloop()
```

Output:

A label + input box appear for user to type name.



(4) Text Widget

For multi-line text input.

```
from tkinter import *

root = Tk()
text = Text(root, height=5, width=30)
text.pack()
root.mainloop()
```



(5) Checkbutton Widget

Used for options that can be ON or OFF.

```
from tkinter import *

root = Tk()
check = Checkbutton(root, text="I agree to terms and conditions")
check.pack()
root.mainloop()
```



(6) Radiobutton Widget

Used for selecting **one option** among many.

```
from tkinter import *

root = Tk()
Label(root, text="Select your gender:").pack()

v = IntVar()
Radiobutton(root, text="Male", variable=v, value=1).pack()
Radiobutton(root, text="Female", variable=v, value=2).pack()

root.mainloop()
```



(7) Listbox Widget

Shows a list of options.

```
from tkinter import *

root = Tk()
listbox = Listbox(root)
listbox.insert(1, "Python")
```

```
listbox.insert(2, "C++")
listbox.insert(3, "Java")
listbox.pack()
root.mainloop()
```

(8) Frame Widget

Used to group multiple widgets.

```
from tkinter import *

root = Tk()
frame = Frame(root)
frame.pack()

Label(frame, text="Inside Frame").pack()
Button(frame, text="Click").pack()

root.mainloop()
```

5. Example: Simple GUI Program (Add Two Numbers)

```
from tkinter import *

def add_numbers():
    num1 = int(entry1.get())
    num2 = int(entry2.get())
    result_label.config(text=f"Result: {num1 + num2}")

root = Tk()
root.title("Addition Program")

Label(root, text="Enter First Number:").pack()
entry1 = Entry(root)
entry1.pack()

Label(root, text="Enter Second Number:").pack()
entry2 = Entry(root)
entry2.pack()

Button(root, text="Add", command=add_numbers).pack()
result_label = Label(root, text="Result: ")
result_label.pack()
```

```
root.mainloop()
```

Output:

A small window takes two numbers → when you click “Add”, it shows the sum.

6. Example: Simple Calculator (Mini Project)

```
from tkinter import *

def click(event):
    text = event.widget.cget("text")
    if text == "=":
        try:
            value = eval(str(screen.get()))
            screen.set(value)
        except Exception as e:
            screen.set("Error")
    elif text == "C":
        screen.set("")
    else:
        screen.set(screen.get() + text)

root = Tk()
root.title("Simple Calculator")

screen = StringVar()
entry = Entry(root, textvar=screen, font="Arial 20 bold")
entry.pack()

buttons = [
    ["7", "8", "9", "+"],
    ["4", "5", "6", "-"],
    ["1", "2", "3", "*"],
    ["C", "0", "=", "/"]
]

for row in buttons:
    frame = Frame(root)
    frame.pack()
    for btn in row:
        b = Button(frame, text=btn, font="Arial 15", width=5,
                    command=lambda t=btn: click(type("Event", ()), {"widget": b}))
```

```
b.bind("<Button-1>", click)
b.pack(side=LEFT)

root.mainloop()
```

✅ This creates a working GUI calculator.

7. Geometry Management

Tkinter uses three layout managers to arrange widgets:

Manager	Description	Example
.pack()	Places widgets one after another	label.pack()
.grid()	Places widgets in table-like structure	label.grid(row=0, column=1)
.place()	Gives exact position	button.place(x=50, y=100)

8. Python Programming with IDE

✅ What is an IDE?

IDE = *Integrated Development Environment*

It provides tools to **write**, **run**, and **debug** Python programs easily.

Common IDEs for Python

IDE	Description	Best For
IDLE	Default Python IDE (comes built-in)	Beginners
PyCharm	Professional IDE with auto-suggestions	Projects
VS Code	Lightweight and popular editor	Students
Jupyter Notebook	Interactive coding + visualization	Data Science
Spyder	Used for scientific coding	Data Analysis

Why Use an IDE?

- Syntax highlighting (colors for code)
- Debugging tools (find and fix errors easily)
- Code suggestions
- File and project management
- Run GUI & scripts smoothly

Example: Running Tkinter in IDE

1. Open **VS Code / PyCharm**
2. Write Tkinter code (like above)

3. Run the file → the GUI window appears

✓ IDE helps you visualize and debug GUI easily.

Summary Table

Concept	Description	Example
Tkinter	Python's GUI library	from tkinter import *
Widget	GUI element like button, label	Label(), Button()
Event	Action like click or keypress	command=greet
IDE	Tool to write/run Python code	VS Code, PyCharm
Geometry Managers	Control layout	pack(), grid(), place()

UNIT 5 – Important Questions & Answers

Part A – Python Packages (NumPy, Pandas, Matplotlib)

Q1. What is NumPy? Why is it used?

Answer:

- **NumPy** stands for *Numerical Python*.
- It is used for **fast mathematical operations** on large data.
- It provides a **powerful N-dimensional array object** called ndarray.

Example:

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(a.mean())    # Output: 2.5
```

Common Use: Data Science, Machine Learning, and Mathematical computations.

Q2. What are NumPy arrays? How are they better than Python lists?

Feature	NumPy Array	Python List
Speed	Faster	Slower
Type	Same data type	Different types allowed
Memory	Less memory	More memory
Operations	Vectorized	Need loops

Example:

```
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
print(a + b) # [5 7 9]
```

Q3. Write a Python program to create a 2D NumPy array and find its shape and size.

Answer:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Shape:", arr.shape)
print("Size:", arr.size)
```

Output:

Shape: (2, 3)
Size: 6

Q4. What is Pandas? Why is it used in Data Science?

Answer:

- **Pandas** = *Python Data Analysis Library*
- It is used for **data manipulation and analysis**.
- It provides two main data structures:
 - Series (1D)
 - DataFrame (2D table-like data)

Example:

```
import pandas as pd
data = {'Name': ['Gautam', 'Neeraj'], 'Marks': [88, 92]}
df = pd.DataFrame(data)
print(df)
```

Output:

```
   Name  Marks
0  Gautam    88
1  Neeraj    92
```

Q5. What is the difference between Series and DataFrame in Pandas?

Feature	Series	DataFrame
Dimension	1D	2D
Structure	Like a column	Like an Excel sheet
Example	pd.Series([1,2,3])	pd.DataFrame({'A':[1,2], 'B':[3,4]})

Q6. Write a program to read and display data from a CSV file using Pandas.

Answer:

```
import pandas as pd
df = pd.read_csv("students.csv")
print(df.head())
```

Explanation:

`read_csv()` reads data from CSV and converts it into a DataFrame.

Q7. What is Matplotlib?

Answer:

- **Matplotlib** is a Python library used to **plot graphs and visualizations**.
- It helps visualize data using line, bar, histogram, scatter plots, etc.

Q8. Write a program to plot a simple line graph using Matplotlib.

Answer:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y)
plt.title("Simple Line Graph")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.show()
```

Output:

Displays a line graph of $y = 2x$.

Q9. Write a program to plot a bar chart in Python.

Answer:

```
import matplotlib.pyplot as plt

names = ['A', 'B', 'C', 'D']
marks = [80, 70, 90, 60]

plt.bar(names, marks)
plt.title("Student Marks")
plt.xlabel("Students")
```

```
plt.ylabel("Marks")  
plt.show()
```

Q10. What is the difference between Matplotlib, NumPy, and Pandas?

Library	Purpose	Example
NumPy	Numeric & array operations	<code>np.array([1,2,3])</code>
Pandas	Data handling (tables)	<code>pd.DataFrame()</code>
Matplotlib	Visualization	<code>plt.plot()</code>



Part B – GUI Programming with Tkinter

Q11. What is Tkinter?

Answer:

Tkinter is Python's **built-in GUI (Graphical User Interface) library** used to create windows, buttons, labels, input boxes, etc.

Q12. Write a simple Tkinter program to display “Hello World”.

Answer:

```
from tkinter import *  
  
root = Tk()  
root.title("First GUI")  
Label(root, text="Hello World!").pack()  
root.mainloop()
```

Q13. What is a widget in Tkinter? Name a few widgets.

Answer:

A **widget** is a GUI element in Tkinter.

Examples:

- Label
- Button
- Entry
- Checkbutton
- Radiobutton
- Text
- Frame

Q14. Write a program in Tkinter to create a button and perform an action on click.

Answer:

```
from tkinter import *

def greet():
    print("Hello Gautam!")

root = Tk()
Button(root, text="Click Me", command=greet).pack()
root.mainloop()
```

Output:

When the button is clicked, "Hello Gautam!" prints on the console.

Q15. What is the use of `mainloop()` in Tkinter?

Answer:

- It is the **infinite event loop** that keeps the GUI window running.
- Without it, the window will close immediately.

Q16. Write a Tkinter program to take input from the user.

Answer:

```
from tkinter import *

root = Tk()
Label(root, text="Enter your name:").pack()
entry = Entry(root)
entry.pack()
Button(root, text="Submit").pack()
root.mainloop()
```

Q17. What is a geometry manager? Name them.

Answer:

Geometry managers control **where widgets are placed** in the window.

- `pack()` → automatic stacking
- `grid()` → row-column layout
- `place()` → fixed coordinates

Q18. Write a program to make a simple addition GUI.

Answer:

```

from tkinter import *

def add():
    a = int(e1.get())
    b = int(e2.get())
    result.config(text=f"Sum = {a+b}")

root = Tk()
Label(root, text="Enter first number:").pack()
e1 = Entry(root); e1.pack()
Label(root, text="Enter second number:").pack()
e2 = Entry(root); e2.pack()
Button(root, text="Add", command=add).pack()
result = Label(root, text="Result = ")
result.pack()
root.mainloop()

```



Part C – Python Programming with IDE

Q19. What is an IDE?

Answer:

- **IDE (Integrated Development Environment)** is a software that provides tools to **write, debug, and run code** easily.
- Example: IDLE, PyCharm, VS Code, Jupyter Notebook.

Q20. Name some popular Python IDEs and their uses.

IDE	Description	Best For
IDLE	Default Python IDE	Beginners
PyCharm	Feature-rich IDE	Large projects
VS Code	Lightweight and powerful	Students & projects
Jupyter Notebook	Interactive coding	Data Science
Spyder	Scientific computing	Data analysis

Q21. Why is using an IDE better than a text editor?

Answer:

Feature	IDE	Text Editor
Auto-completion	✅ Yes	❌ No
Debugging	✅ Easy	❌ Manual

Visualization	✔ Supported	✗ No
Multiple file handling	✔ Yes	⚠ Limited

Q22. What are the advantages of using IDE for GUI Programming?

Answer:

- Easy debugging of Tkinter apps
- Instant GUI preview
- Auto-suggestions for widget functions
- Code formatting and error hints

NOTES BY - [GKM](#)